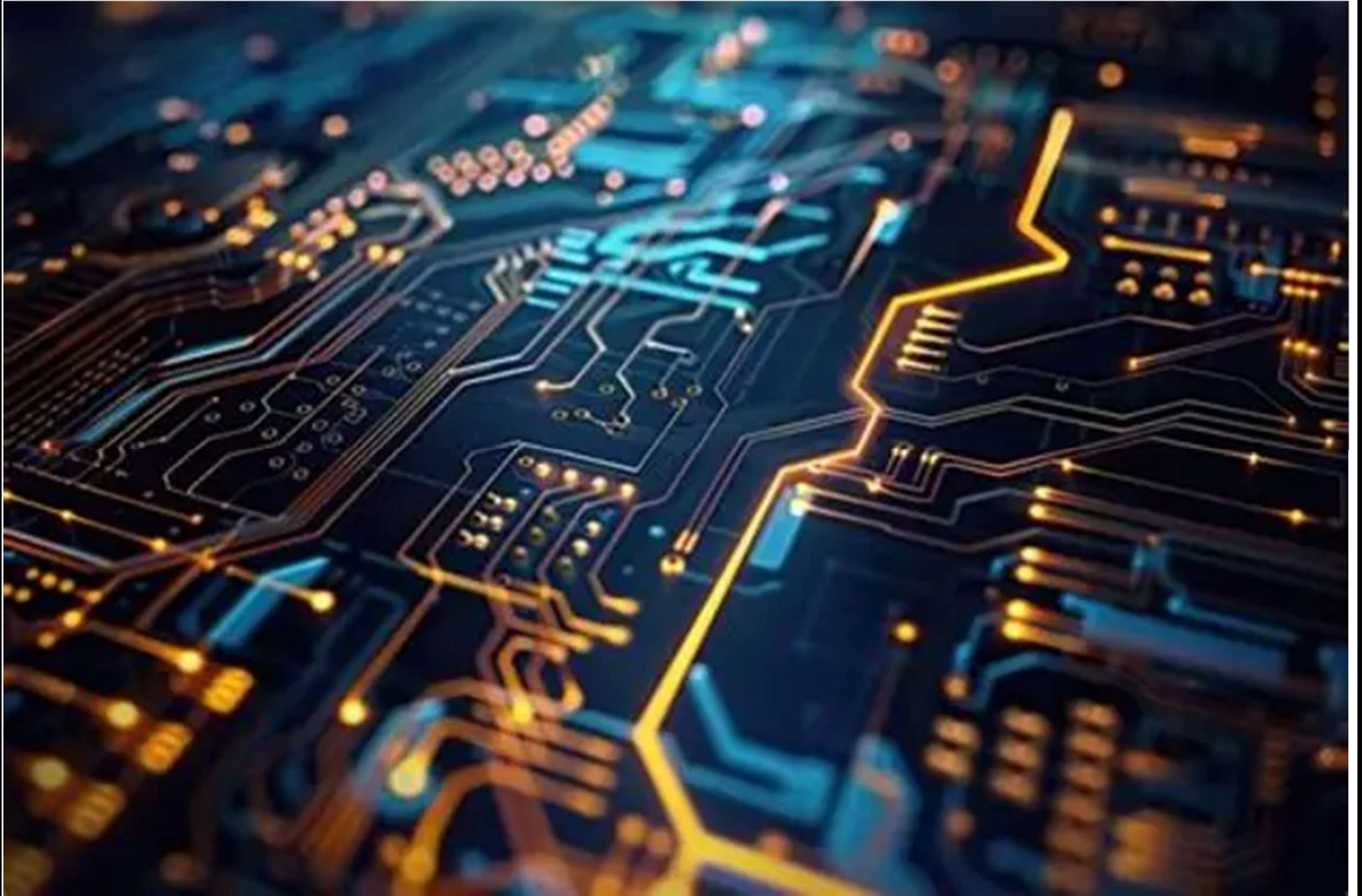


Lab: 2-bit Full Adder



Name: Ahmed Wael Sadek El
Sawy

Lab: 2-bit Full Adder

2-bit Full Adder

half_adder.v

```
module half_adder (  
    input a,  
    input b,  
    output sum,  
    output cout  
);  
    xor (sum, a, b);  
    and (cout, a, b);  
endmodule
```

full_adder_1bit.v

```
module full_adder (  
    input a,  
    input b,  
    input cin,  
    output sum,  
    output cout  
);  
    wire ha1_sum, ha1_cout, ha2_cout;  
  
    half_adder HA1 (.a(a), .b(b), .sum(ha1_sum), .cout(ha1_cout));  
    half_adder HA2 (.a(ha1_sum), .b(cin), .sum(sum), .cout(ha2_cout));  
  
    or (cout, ha1_cout, ha2_cout);  
endmodule
```

full_adder_2bit.v

```
module full_adder_2bit (  
    input [1:0] a,  
    input [1:0] b,  
    input cin,  
    output [1:0] sum,  
    output cout  
);  
    wire c1;  
  
    // Bit 0  
    full_adder FA0 (  
        .a(a[0]), .b(b[0]), .cin(cin),  
        .sum(sum[0]), .cout(c1)
```

Lab: 2-bit Full Adder

```
);  
  
// Bit 1  
full_adder FA1 (  
    .a(a[1]), .b(b[1]), .cin(c1),  
    .sum(sum[1]), .cout(cout)  
);  
endmodule
```

fa_2bit_gate.v

```
module fa_2bit_gate (  
    input [1:0] a,  
    input [1:0] b,  
    input cin,  
    output [1:0] sum,  
    output cout  
);  
    wire w1, w2, w3;  
    wire c1;  
    wire w4, w5, w6;  
  
    xor (w1, a[0], b[0]);  
    xor (sum[0], w1, cin);  
    and (w2, a[0], b[0]);  
    and (w3, w1, cin);  
    or (c1, w2, w3);  
  
    xor (w4, a[1], b[1]);  
    xor (sum[1], w4, c1);  
    and (w5, a[1], b[1]);  
    and (w6, w4, c1);  
    or (cout, w5, w6);  
  
endmodule
```

fa_2bit_behavioral.v

```
module fa_2bit_behavioral (  
    input [1:0] a,  
    input [1:0] b,  
    input cin,  
    output [1:0] sum,  
    output cout  
);
```

Lab: 2-bit Full Adder

```
assign {cout, sum} = a + b + cin;
```

```
endmodule
```

tb_2bit_full_adders.v

```
module tb_2bit_full_adders;
```

```
    // Testbench signals
```

```
    reg [1:0] tb_a;
```

```
    reg [1:0] tb_b;
```

```
    reg tb_cin;
```

```
    // Outputs from DUTs
```

```
    wire [1:0] gate_sum, struct_sum, behav_sum;
```

```
    wire gate_cout, struct_cout, behav_cout;
```

```
    // Self-checking variables
```

```
    reg [1:0] exp_sum;
```

```
    reg exp_cout;
```

```
    integer error_count = 0;
```

```
    integer i;
```

```
    // Instantiate the Gate-Level 2-Bit FA
```

```
    fa_2bit_gate DUT_GATE (  
        .a(tb_a), .b(tb_b), .cin(tb_cin),  
        .sum(gate_sum), .cout(gate_cout)  
    );
```

```
    // Instantiate the Structural 2-Bit FA
```

```
    fa_2bit_structural DUT_STRUCT (  
        .a(tb_a), .b(tb_b), .cin(tb_cin),  
        .sum(struct_sum), .cout(struct_cout)  
    );
```

```
    // Instantiate the Behavioral 2-Bit FA
```

```
    fa_2bit_behavioral DUT_BEHAV (  
        .a(tb_a), .b(tb_b), .cin(tb_cin),  
        .sum(behav_sum), .cout(behav_cout)  
    );
```

```
    // Stimulus and Self-Checking Task
```

```
    task check_results;
```

```
        begin
```

```
            // Calculate the expected true mathematical sum
```

```
            {exp_cout, exp_sum} = tb_a + tb_b + tb_cin;
```

Lab: 2-bit Full Adder

```
// Check Gate Level
if ({gate_cout, gate_sum} != {exp_cout, exp_sum}) begin
    $display("ERROR (Gate): A=%b, B=%b, Cin=%b | Expected={%b,%b},
Got={%b,%b}",
            tb_a, tb_b, tb_cin, exp_cout, exp_sum, gate_cout,
gate_sum);
    error_count = error_count + 1;
end

// Check Structural Level
if ({struct_cout, struct_sum} != {exp_cout, exp_sum}) begin
    $display("ERROR (Struct): A=%b, B=%b, Cin=%b | Expected={%b,%b},
Got={%b,%b}",
            tb_a, tb_b, tb_cin, exp_cout, exp_sum, struct_cout,
struct_sum);
    error_count = error_count + 1;
end

// Check Behavioral Level
if ({behav_cout, behav_sum} != {exp_cout, exp_sum}) begin
    $display("ERROR (Behav): A=%b, B=%b, Cin=%b | Expected={%b,%b},
Got={%b,%b}",
            tb_a, tb_b, tb_cin, exp_cout, exp_sum, behav_cout,
behav_sum);
    error_count = error_count + 1;
end
end
endtask

initial begin
    $display("Starting 2-Bit Full Adder Tests...");

    // Loop through all 32 possible combinations of A(2), B(2), and Cin(1)
    for (i = 0; i < 32; i = i + 1) begin
        {tb_a, tb_b, tb_cin} = i;
        #10;                // Wait for combinational logic to settle
        check_results();    // Call the self-checking validation
    end

    // Final Report

    if (error_count == 0)
        $display("SUCCESS: All 3 modules passed 32/32 tests with 0 errors!");
```

Lab: 2-bit Full Adder

```
        else
            $display("FAILED: Testbench completed with %0d total errors.",
error_count);

        $stop;
    end

endmodule
```

Testing 2 bit Full Adder

